

Name : Chetan Bapurao Jadhav

Prn no : 202501040051

Division : CS-1

Batch : C13

Course : EDS

Essential of Data Science Laboratory

Practical 4:

4.1.1 Pandas - Series Creation and Manipulation:

The screenshot displays a coding environment with the following components:

- Header:** CODETANTRA logo, Home, Learn Anywhere, user ID 202501040051@mitaoe.ac.in, Support, and Logout.
- Problem Statement:** 4.1.1. Pandas - series creation and manipulation. Write a Python program that takes a list of numbers from the user, creates a Pandas series from it, and then calculates the mean of even and odd numbers separately using the `groupby` and `mean()` operations. Hint: You can group the Series based on whether each number is even or odd.
- Input Format:** The user should enter a list of numbers separated by space when prompted.
- Output Format:** The program should display the mean of even and odd numbers separately. Each mean value should be displayed with a label indicating whether it corresponds to even or odd numbers.
- Sample Test Cases:** A section for viewing test cases.
- Code Editor:** Contains the following Python code:

```
1 import pandas as pd
2
3 # Take inputs from the user to create a list of numbers
4 numbers = list(map(int, input().split()))
5
6 # Create a Pandas series from the list of numbers
7 series = pd.Series(numbers)
8 # Grouping by even and odd numbers and calculating the mean
9 grouped = series.groupby(series % 2 == 0).mean()
10
11 # Display the mean of even and odd numbers with labels
12 grouped.index = ['Even' if is_even else 'Odd' for is_even in
```
- Execution Results:**
 - Average time: 0.014 s, Maximum time: 0.036 s.
 - 3 out of 3 shown test case(s) passed.
 - 3 out of 3 hidden test case(s) passed.
 - Test case 1 (36 ms) is shown with a Debug button.
 - Expected output:** 1 2 3 4 5 6 7 8 9 10, Mean of even and odd numbers: 5.0, Odd: 5.0, Even: 6.0, dtype: float64.
 - Actual output:** 1 2 3 4 5 6 7 8 9 10, Mean of even and odd numbers: 5.0, Odd: 5.0, Even: 6.0, dtype: float64.
- Footer:** Terminal, Test cases, Prev, Reset, Submit, Next buttons.

4.1.2 Dictionary to dataframe:

[Home](#)
[Learn Anywhere](#)

202501040051@mitaoe.ac.in
Support
Logout

4.1.2. Dictionary to dataframe
38.00

A dictionary of lists has been provided to you in the editor. Create a DataFrame from the dictionary of lists and perform the listed operations, then display the DataFrame before and after each manipulation.

Create the DataFrame:

- Convert the dictionary to a Pandas DataFrame.

Add a new row:

- Take inputs from the user for the new row data (name, age).
- Add the new row to the DataFrame.
- Display the DataFrame after adding the new row.

Modify a row:

- Modify a specific row by changing the age. Take the row index and new age value from the user.
- Display the DataFrame after modifying the row.

Delete a row:

- Take the row index to be deleted from the user.
- Remove the specified row.
- Display the DataFrame after deleting the row.

Add a new column:

- Add a column **Gender** with values taken from the user.
- Display the DataFrame after adding the new column.

Modify a column:

- Convert names to uppercase.

datafram...

```

1 import pandas as pd
2
3 # Provided dictionary of lists
4 data = {
5     'Name': ['Alice', 'Bob', 'Charlie'],
6     'Age': [25, 30, 35],
7 }
8
9 # Convert the dictionary to a DataFrame
10 df = pd.DataFrame(data)
11
12 # Display the original DataFrame
13 print("Original DataFrame:")
14 print(df)
15
16 # Adding a new row
17 new_name = input("New name: ")
18 new_age = int(input("New age: "))
19 df.loc[len(df)] = [new_name, new_age]
20
21 # Display the DataFrame after adding a new row
22 print("After adding a row:\n", df)
23
24 # Modifying a row
25 row_modify = int(input("Index of row to modify: "))

```

Terminal
Test cases

< Prev
Reset
Submit
Next >

Average time
0.110 s
110.00 ms

Maximum time
0.122 s
122.00 ms

1 out of 1 shown test case(s) passed

1 out of 1 hidden test case(s) passed

Test case 1
122 ms
Debug

Expected output

Actual output

Original DataFrame:

```

.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
New name: Susan
New age: 29
After adding a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3...Susan...29
Index of row to modify: 0
New age: 27
After modifying a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3...Susan...27

```

Original DataFrame:

```

.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
New name: Susan
New age: 29
After adding a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3...Susan...29
Index of row to modify: 0
New age: 27
After modifying a row:
.....Name..Age
0....Alice...25
1....Bob...30
2..Charlie...35
3...Susan...27

```

Terminal
Test cases

4.1.3 Student Information:

CODETANTRAHomeLearn Anywhere202501040051@mitaoe.ac.inSupportLogout

4.1.3. Student Information17:13

Write a program to read a text file containing student information (name, age, and grade) using Pandas. Perform the following tasks:

- Display the first five rows of the data frame.
- Calculate the average age of the students (limit the average age up to 2 decimal places).
- Filter out the students who have a grade above a certain threshold (consider the threshold grade is 'B').

Note:
Refer to the displayed test cases for better understanding.

Sample Test Cases+

studentin...

```
1 import pandas as pd
2
3 # Read the text file into a DataFrame
4 file = input()
5 data = pd.read_csv(file, sep="\s+", header=None, names=["Name", "Age",
6 "Grade"])
7 # write your code here..
8 print("First five rows:")
9 print(data.head())
10
11 average_age = round(data['Age'].mean(), 2)
12 print(f"Average age: {average_age}")
13
14 filter_students = data[data["Grade"]<="B"]
15 print("Students with a grade up to B")
16 print(filter_students)
```

Average time0.050 s49.50 msMaximum time0.072 s72.00 ms

1 out of 1 shown test case(s) passed1 out of 1 hidden test case(s) passed

Test case 172 msDebug

Expected outputActual output

studentdata.txtstudentdata.txt

TerminalTest cases

< PrevResetSubmitNext>

4.2.1 Month with Highest total Sales:

CODETANTRAHomeLearn Anywhere202501040051@mitaoe.ac.inSupportLogout

4.2.1. Month with the Highest Total Sales16:03

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by Month and calculate the total sales for each month.
- Find the month with the highest total sales and display it.
- Also, display the total sales for the best month.

Sample Data:

```
Date,Product,Quantity,Price,City
2025-01-01,Product A,5,20,New York
2025-01-01,Product B,3,15,Los Angeles
2025-01-02,Product A,7,20,New York
2025-01-02,Product C,4,30,Chicago
2025-01-03,Product B,2,15,Chicago
2025-01-03,Product A,8,20,Los Angeles
2025-01-04,Product C,6,30,New York
2025-01-04,Product B,5,15,Los Angeles
2025-01-05,Product A,3,20,Chicago
2025-01-05,Product C,10,30,Los Angeles
```

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases+

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9
10 # Find the month with the highest total sales
11 best_month =
12 (df.iloc[:,2]*df.iloc[:,3]).groupby(pd.to_datetime(df.iloc[:,0]).dt.strftime('%Y-%m')).sum().idxmax()
13 highest_sales =
14 (df.iloc[:,2]*df.iloc[:,3]).groupby(pd.to_datetime(df.iloc[:,0]).dt.strftime('%Y-%m')).sum().max()
15
16 print(f"Best month: {best_month}")
17 print(f"Total sales: ${highest_sales:.2f}")
```

TerminalTest cases

< PrevResetSubmitNext>

Average time

0.048 s

48.00 ms

Maximum time

0.095 s

95.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

95 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best month: 2025-01

Best month: 2025-01

Total sales: \$1210.00

Total sales: \$1210.00

4.2.2 Best Selling Product:

CODETANTRA

Home Learn Anywhere

202501040051@mitaoe.ac.in Support Logout

4.2.2. Best Selling Product

01:48

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Find the product that sold the most in terms of quantity sold.
- Display the product that sold the most and the total quantity sold for that product.

Sample Data:

Date,Product,Quantity,Price,City

2025-01-01,Product A,5,20,New York

2025-01-01,Product B,3,15,Los Angeles

2025-01-02,Product A,7,20,New York

2025-01-02,Product C,4,30,Chicago

2025-01-03,Product B,2,15,Chicago

2025-01-03,Product A,8,20,Los Angeles

2025-01-04,Product C,6,30,New York

2025-01-04,Product B,5,15,Los Angeles

2025-01-05,Product A,3,20,Chicago

2025-01-05,Product C,10,30,Los Angeles

Note:

The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

Submit

1 import pandas as pd

2

3 # Prompt the user for the file name

4 file_name = input()

5

6 # Load the data

7 df = pd.read_csv(file_name)

8

9 # Find the product with the highest total quantity sold

10 best_product = df.groupby(df.iloc[:,1])[df.iloc[:,2].name].sum().idxmax()

11 highest_quantity = df.groupby(df.iloc[:,1])[df.iloc[:,2].name].sum().max()

Average time

0.035 s

34.67 ms

Maximum time

0.064 s

64.00 ms

1 out of 1 shown test case(s) passed

2 out of 2 hidden test case(s) passed

Test case 1

64 ms

Debug

Expected output

Actual output

sales_data.csv

sales_data.csv

Best selling product: Product A

Best selling product: Product A

Total quantity sold: 23

Total quantity sold: 23

Terminal

Test cases

4.2.3 City that Sold Most Products:

CODETANTRA Home Learn Anywhere

202501040051@mitaoe.ac.in Support Logout

4.2.3. City that Sold the Most Products 12:50

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the columns: Date, Product, Quantity, Price, and City.
- Group the data by City and calculate the total quantity of products sold for each city.
- Find the city that sold the most products (based on the total quantity sold).

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Note:
The data cannot be displayed in the file. You can refer to the sample data provided for insights.

Sample Test Cases

monthFor...

```
1 import pandas as pd
2
3 # Prompt the user for the file name
4 file_name = input()
5
6 # Load the data
7 df = pd.read_csv(file_name)
8
9 best_city = df.groupby('City')['Quantity'].sum().idxmax()
10
11 # Display the result
12 print(f"City sold the most products: {best_city}")
13
```

Average time: 0.027 s (27.33 ms) Maximum time: 0.056 s (56.00 ms) 1 out of 1 shown test case(s) passed 2 out of 2 hidden test case(s) passed

Test case 1 56 ms Debug

Expected output	Actual output
sales_data.csv	sales_data.csv
City sold the most products: Los Angeles	City sold the most products: Los Angeles

Terminal Test cases

4.2.4 Most Frequently Sold Product Pairs:

CODETANTRA Home Learn Anywhere

202501040051@mitaoe.ac.in Support Logout

4.2.4. Most Frequently Sold Product Pairs 04:38

Write a Python program that takes the file name of a CSV file as input, reads the data, and performs the following operations:

- The CSV file contains the following columns: Date, Product, Quantity, Price, and City.
- For each date, find all pairs of products that were sold together (i.e., two products sold on the same date).
- Output the product pair/s that was sold most frequently.

Sample Data:

Date	Product	Quantity	Price	City
2025-01-01	Product A	5	20	New York
2025-01-01	Product B	3	15	Los Angeles
2025-01-02	Product A	7	20	New York
2025-01-02	Product C	4	30	Chicago
2025-01-03	Product B	2	15	Chicago
2025-01-03	Product A	8	20	Los Angeles
2025-01-04	Product C	6	30	New York
2025-01-04	Product B	5	15	Los Angeles
2025-01-05	Product A	3	20	Chicago
2025-01-05	Product C	10	30	Los Angeles

Explanation:
Transactions:

- 2025-01-01: Product A, Product B
- 2025-01-02: Product A, Product C
- 2025-01-03: Product B, Product A
- 2025-01-04: Product C, Product B
- 2025-01-05: Product A, Product C

frequentl...

```
1 import pandas as pd
2 from itertools import combinations
3 from collections import Counter
4
5 # Prompt user to input the file name
6 file_name = input()
7
8 # Read data from the specified CSV file
9 df = pd.read_csv(file_name)
10
11 # Step 1: Group products by Date
12 grouped = df.groupby('Date')['Product'].apply(lambda x: list(set(x)))
13
14 # Step 2: Generate product pairs
15 pairs = []
16 for products in grouped:
17     pairs += [tuple(sorted(pair)) for pair in combinations(products, 2)]
18
19 # Step 3: Count frequency of each pair
20 pair_counts = Counter(pairs)
21
22 # Step 4: Find maximum frequency
23 max_count = max(pair_counts.values())
24
25 # Step 5: Print only most frequent pairs
```

Terminal Test cases

Average time
0.025 s
24.67 ms

Maximum time
0.039 s
39.00 ms

1 out of 1 shown test case(s) passed
2 out of 2 hidden test case(s) passed

Test case 1
22 ms
Debug

Expected output
sales_data.csv
Product-A-and-Product-B:-2-times
Product-A-and-Product-C:-2-times

Actual output
sales_data.csv
Product-A-and-Product-B:-2-times
Product-A-and-Product-C:-2-times

4.2.5 Titanic Dataset Analysis:

CODETANTRA
Home
Learn Anywhere
202501040051@mitaoe.ac.in
Support
Logout

4.2.5. Titanic Dataset Analysis and Data Cleaning
06:41

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset. For each question, perform necessary data cleaning, transformations, and calculations as required.

- Display the first 5 rows of the dataset.
- Display the last 5 rows of the dataset.
- Get the shape of the dataset (number of rows and columns).
- Get a summary of the dataset (using `.info()`).
- Get basic statistics (mean, standard deviation, etc.) of the dataset using `.describe()`.
- Check for missing values and display the count of missing values for each column.
- Fill missing values in the 'Age' column with the median age.
- Fill missing values in the 'Embarked' column with the most frequent value (`mode()`).
- Drop the 'Cabin' column due to many missing values.
- Create a new column, 'FamilySize' by adding the 'SibSp' and 'Parch' columns.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer
titanicDat...
Submit
Debugger

```

1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6
7 # 1. Display the first 5 rows of the dataset
8 print(data.head())
9
10 # 2. Display the last 5 rows of the dataset
11 print(data.tail())
12
13 # 3. Get the shape of the dataset
14 print(data.shape)
15
16 # 4. Get a summary of the dataset (info)
17 print(data.info())
18
19 # 5. Get basic statistics of the dataset
20 print(data.describe())
21
22 # 6. Check for missing values
23 print(data.isnull().sum())
24
25 # 7. Fill missing values in the 'Age' column with the median age

```

Terminal
Test cases

Average time
0.298 s
298.00 ms

Maximum time
0.298 s
298.00 ms

1 out of 1 shown test case(s) passed

Test case 1
298 ms
Debug

Expected output
...PassengerId...Survived...Pclass...Fare...Cabin...Embarked...
0...1...0...3...7.2500...NaN...S...
1...2...1...1...71.2833...C85...C...
2...3...1...3...7.9250...NaN...S...
3...4...1...1...53.1000...C123...S...
4...5...0...3...8.0500...NaN...S...
[5 rows x 12 columns]
...PassengerId...Survived...Pclass...Fare...Cabin...Embarked...
886...887...0...2...13.00...NaN...S...
887...888...1...1...30.00...B42...S...
888...889...0...3...23.45...NaN...S...
889...890...1...1...30.00...C148...C...
890...891...0...3...7.75...NaN...Q...
[5 rows x 12 columns]
(891, 12)
<class 'pandas.core.frame.DataFrame'>

Actual output
...PassengerId...Survived...Pclass...Fare...Cabin...Embarked...
0...1...0...3...7.2500...NaN...S...
1...2...1...1...71.2833...C85...C...
2...3...1...3...7.9250...NaN...S...
3...4...1...1...53.1000...C123...S...
4...5...0...3...8.0500...NaN...S...
[5 rows x 12 columns]
...PassengerId...Survived...Pclass...Fare...Cabin...Embarked...
886...887...0...2...13.00...NaN...S...
887...888...1...1...30.00...B42...S...
888...889...0...3...23.45...NaN...S...
889...890...1...1...30.00...C148...C...
890...891...0...3...7.75...NaN...Q...
[5 rows x 12 columns]
(891, 12)
<class 'pandas.core.frame.DataFrame'>

4.2.6 Titanic Dataset Analysis:

CODETANTRA

Home Learn Anywhere

202501040051@mitaoe.ac.in Support Logout

4.2.6. Titanic Dataset Analysis and Data Cleaning - 202:51

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Create a new column 'IsAlone' which is 1 if the passenger is alone (FamilySize = 0), otherwise 0.
2. Convert the 'Sex' column to numeric values (male: 0, female: 1).
3. One-hot encode the 'Embarked' column, dropping the first category.
4. Get the mean age of passengers.
5. Get the median fare of passengers.
6. Get the number of passengers by class.
7. Get the number of passengers by gender.
8. Get the number of passengers by survival status.
9. Calculate the survival rate of passengers.
10. Calculate the survival rate by gender.

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer

titanicDat...

Submit

Debugger

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7
8 # 1. IsAlone
9 data['IsAlone'] = data['FamilySize'].apply(lambda x: 1 if x == 0 else 0)
10
11 # 2. Convert Sex
12 data['Sex'] = data['Sex'].map({'male': 0, 'female': 1})
13
14 # 3. One-hot encode Embarked
15 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
16
17 # 4. Mean Age
18 print(data['Age'].mean())
19
20 # 5. Median Fare
21 print(data['Fare'].median())
22
23 # 6. Passengers by Class
24 print(data['Pclass'].value_counts())
25
```

Terminal Test cases

< Prev Reset Submit Next >

Average time0.143 s143.00 ms

Maximum time0.143 s143.00 ms

1 out of 1 shown test case(s) passed

Test case 1143 msDebug

Expected output

Actual output

29.69911764705882

29.69911764705882

14.4542

14.4542

3 --- 491

3 --- 491

1 --- 216

1 --- 216

2 --- 184

2 --- 184

Name: Pclass, dtype: int64

Name: Pclass, dtype: int64

0 --- 577

0 --- 577

1 --- 314

1 --- 314

Name: Sex, dtype: int64

Name: Sex, dtype: int64

0 --- 549

0 --- 549

1 --- 342

1 --- 342

Name: Survived, dtype: int64

Name: Survived, dtype: int64

0.3838383838383838

0.3838383838383838

Sex

Sex

0 --- 0.188908

0 --- 0.188908

1 --- 0.742038

1 --- 0.742038

Name: Survived, dtype: float64

Name: Survived, dtype: float64

Terminal Test cases

< Prev Reset Submit Next >

4.2.7 Titanic Dataset Analysis:

CODETANTRA Home Learn Anywhere

202501040051@mitaoe.ac.in Support Logout

4.2.7. Titanic Dataset Analysis and Data Cleaning - 3

12:52

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Calculate the survival rate by class.
2. Calculate the survival rate by embarkation location (Embarked_S).
3. Calculate the survival rate by family size (FamilySize).
4. Calculate the survival rate by being alone (IsAlone).
5. Get the average fare by passenger class (Pclass).
6. Get the average age by passenger class (Pclass).
7. Get the average age by survival status (Survived).
8. Get the average fare by survival status (Survived).
9. Get the number of survivors by class (Pclass).
10. Get the number of non-survivors by class (Pclass).

The Titanic dataset contains columns as shown below,

PassengerId	Survived	Pclass	Name	Sex	Age	SibSp	Parch	Ticket	Fare	Cabin	Embarked

Sample Data:

Explorer

titanicDat...

Submit

Debugger

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data['FamilySize'] = data['SibSp'] + data['Parch']
7 data['IsAlone'] = np.where(data['FamilySize'] > 0, 0, 1)
8 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
9
10 # 1. Survival rate by class
11 print(data.groupby('Pclass')['Survived'].mean())
12
13 # 2. Survival rate by embarked location (Embarked_S)
14 print(data.groupby('Embarked_S')['Survived'].mean())
15
16 # 3. Survival rate by family size
17 print(data.groupby('FamilySize')['Survived'].mean())
18
19 # 4. Survival rate by being alone
20 print(data.groupby('IsAlone')['Survived'].mean())
21
22 # 5. Average fare by class
23 print(data.groupby('Pclass')['Fare'].mean())
24
25 # 6. Average age by class
```

Terminal Test cases

< Prev Reset Submit Next >

Average time
0.155 s
155.00 ms

Maximum time
0.155 s
155.00 ms

1 out of 1 shown test case(s) passed

Test case 1 155 ms Debug

Expected output

Actual output

Pclass	Pclass
1...0.629630	1...0.629630
2...0.472826	2...0.472826
3...0.242363	3...0.242363
Name: Survived, dtype: float64	Name: Survived, dtype: float64
Embarked_S	Embarked_S
0...0.506073	0...0.506073
1...0.336957	1...0.336957
Name: Survived, dtype: float64	Name: Survived, dtype: float64
FamilySize	FamilySize
0...0.303538	0...0.303538
1...0.552795	1...0.552795
2...0.578431	2...0.578431
3...0.724138	3...0.724138
4...0.200000	4...0.200000
5...0.136364	5...0.136364
6...0.333333	6...0.333333
7...0.000000	7...0.000000

4.2.8 Titanic Dataset Analysis:

4.2.8. Titanic Dataset Analysis and Data Cleaning - 4

01:24

🔍 🌙 📄 🗑

You are provided with the Titanic dataset containing information about passengers on the Titanic. Your task is to write Python code to answer the following questions based on the dataset.

1. Get the number of survivors by gender (Sex).

2. Get the number of non-survivors by gender (Sex).

3. Get the number of survivors by embarkation location (Embarked_S).

4. Get the number of non-survivors by embarkation location (Embarked_S).

5. Calculate the percentage of children (Age < 18) who survived.

6. Calculate the percentage of adults (Age >= 18) who survived.

7. Get the median age of survivors.

8. Get the median age of non-survivors.

9. Get the median fare of survivors.

10. Get the median fare of non-survivors.

The Titanic dataset contains columns as shown below,

Pas sen ger id	Sur vive d	Pcla ss	Na me	Sex	Age	Sib Sp	Par ch	Tick et	Fare	Cab in	Em bark ed

Sample Data:

Explorer

titanicDat...

Submit

Debugger

```
1 import pandas as pd
2 import numpy as np
3
4 # Load the Titanic dataset
5 data = pd.read_csv('Titanic-Dataset.csv')
6 data = pd.get_dummies(data, columns=['Embarked'], drop_first=True)
7
8
9 # 1. Number of survivors by gender
10 print(data[data['Survived'] == 1]['Sex'].value_counts())
11
12 # 2. Number of non-survivors by gender
13 print(data[data['Survived'] == 0]['Sex'].value_counts())
14
15 # 3. Number of survivors by embarked location (Embarked_S)
16 print(data[data['Survived'] == 1]['Embarked_S'].value_counts())
17
18 # 4. Number of non-survivors by embarked location (Embarked_S)
19 print(data[data['Survived'] == 0]['Embarked_S'].value_counts())
20
21 # 5. Percentage of children (Age < 18) who survived
22 children = data[data['Age'] < 18]
23 print(children['Survived'].mean())
24
25 # 6. Percentage of adults (Age >= 18) who survived
```

Terminal

Test cases

< Prev

Reset

Submit

Next >

Average time

0.147 s

147.00 ms

Maximum time

0.147 s

147.00 ms

1 out of 1 shown test case(s) passed

Test case 1

147 ms

Debug

⋮

📄

Expected output

Actual output

female.....233

female.....233

male.....109

male.....109

Name: Sex, dtype: int64

Name: Sex, dtype: int64

male.....468

male.....468

female.....81

female.....81

Name: Sex, dtype: int64

Name: Sex, dtype: int64

1....217

1....217

0....125

0....125

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

1....427

1....427

0....122

0....122

Name: Embarked_S, dtype: int64

Name: Embarked_S, dtype: int64

0.5398230088495575

0.5398230088495575

0.3810316139767055

0.3810316139767055

28.0

28.0

28.0

28.0

26.0

26.0

10.5

10.5

